STANDPOINT

A LinkedIn Positioning System

Systems Report · Prepared 2026-09-10

1. What This Project Is

Standpoint is a positioning system for LinkedIn, not a content-generation tool. The unit of work is the position: what a person stands for, professionally, and is willing to defend in public. Every post is a subordinate artifact of that position; it exists to advance a declared stance, not the other way around.

The product's core mechanism is a gate, not a generator. A user builds a belief graph (a brand profile, a set of stances, including at least one genuinely contrarian belief, and explicit anti-positions they refuse to argue), organizes it into up to three content pillars, generates ideas within those pillars, and writes drafts. Before any draft can be published to LinkedIn, it is scored by an adversarial AI reviewer that has no incentive to approve it. Only a passing verdict unlocks publishing, and that check is enforced in the database, not in the UI, so it cannot be bypassed by a disabled button, a direct API call, or a client-side hack.

The product deliberately excludes the growth-hacking toolkit common to LinkedIn tools: no scheduling, no engagement automation, no auto-comments or auto-DMs, no engagement metrics of any kind, and no vanity analytics. It measures whether someone is being distinct and defensible, never whether a post performed.

2. Architecture

2.1 Stack

- Client: React + Vite + TypeScript + Tailwind CSS
- Backend: Supabase (Postgres, Auth, Row Level Security, Edge Functions)
- LLM access: a single Supabase Edge Function, llm-gateway, is the only code path that calls the LLM provider
- LinkedIn: Sign In with LinkedIn (OIDC) for identity; Share on LinkedIn (w_member_social scope) for publishing

2.2 LLM Provider

The Edge Function calls Groq's OpenAI-compatible chat completions API. Groq was chosen deliberately over a paid Anthropic API to avoid an ongoing API bill. The API key is stored as a Supabase secret and never appears in client code, the repository, or any committed file.

2.3 Client / Server Boundary

The browser client only ever talks to two things: Supabase directly (using the RLS-protected anon key) and the llm-gateway Edge Function. It never calls the LLM provider or LinkedIn's token endpoints directly, and it never holds a client secret.

3. Data Model

Nine tables form a strict hierarchy: draft → idea → pillar → stance. Every table has Row Level Security enabled; tables without their own user_id column (stances, anti_positions, pillars, ideas, drafts, review_results, published_posts) are scoped by traversing foreign keys back to brand_profiles.user_id.

Table	Columns
users	id, email
brand_profiles	id, user_id, role_title, icp_title, icp_industry, icp_pains (jsonb)
stances	id, profile_id, statement, type [core | contrarian], evidence
anti_positions	id, profile_id, topic, reason
pillars	id, profile_id, name, stance_id, target_pain (max 3 per profile)
ideas	id, pillar_id, angle, angle_type [contrarian | framework | evidence | story], target_pain
drafts	id, idea_id, body, status [writing | reviewed | published]
review_results	id, draft_id, scores (jsonb), verdict, reasons (jsonb), fixes (jsonb), created_at
published_posts	id, draft_id, linkedin_urn, published_at

4. Non-Negotiable Rules

Five rules define the product and are enforced in Postgres, through triggers, RPCs, and RLS policies, not in application code or the UI. A disabled button is treated as a UX nicety, never as the actual enforcement mechanism.

1. FK chain integrity

A draft cannot exist without an idea; an idea cannot exist without a pillar; a pillar cannot exist without a stance. Enforced with NOT NULL foreign-key columns; no nullable shortcuts.

2. Contrarian-belief gate

A user cannot create a pillar until their brand profile has at least one stance with type = 'contrarian'. Enforced with a Postgres BEFORE INSERT trigger on pillars; direct SQL/Studio inserts are blocked too.

3. Pillar cap

A maximum of three pillars per profile, enforced with a server-side trigger/constraint.

4. Publish gate

Publishing is only possible through the publish_draft(draft_id) SECURITY DEFINER RPC, which checks that the draft's most recent review_results row has verdict = 'PUBLISH' before touching published_posts or flipping drafts.status. RLS denies any direct client insert into published_posts.

5. No engagement metrics, ever

The product never adds, fetches, stores, or displays impressions, likes, reactions, follower counts, or any other engagement metric.

5. The Review Gate

The review gate is the core of the product. A dedicated adversarial system prompt, deliberately separate from any generation prompt, receives the author's belief graph (domains, stances, contrarian beliefs, anti-positions, proprietary frameworks, ICP problems), their previously published posts, and one draft. It has no investment in the draft passing and is instructed never to praise, only to explain.

5.1 Scoring dimensions

Dimension	Scale	What it measures
pillar_fit	0-100, higher is better	Does the draft reinforce a declared domain, or drift?
stance_alignment	0-100, higher is better	Does it advance a specific declared stance? REJECT if < 30.

Dimension	Scale	What it measures
substitutability	0-100, higher is WORSE	Could 500 peers have written this? REJECT if > 75.
evidence_specificity	0-100, higher is better	Concrete number, named case, or lived detail vs. assertion.
ai_slop_risk	0-100, higher is WORSE	Generic AI shapes: listicles, broetry, engagement bait.

5.2 Verdict logic (applied in order)

- REJECT if the draft hits a declared anti-position
- REJECT if stance_alignment < 30 OR substitutability > 75
- REVISE if evidence_specificity < 40 OR ai_slop_risk > 60 OR the draft contradicts a previously published post
- Otherwise, PUBLISH

Only PUBLISH unlocks the publish_draft RPC. REVISE and REJECT both block publishing but carry different meaning to the author: rework versus scrap. Every fix returned by the reviewer must reference something concrete in the author's own material; "add more detail" is treated as a useless fix by design.

6. User Workflow

- Brand profile: role, ICP title/industry, ICP pains, stances (core and contrarian), anti-positions. Can be entered manually or drafted from a structured interview flow and then edited/confirmed.
- Pillars: up to three content domains, each anchored to a stance. Blocked server-side until a contrarian stance exists.
- Ideas: specific angles within a pillar, whether contrarian, framework, evidence, or story.
- Drafts: written from an idea, with the governing pillar, stance, and active anti-positions always visible in the editor, including at 390px mobile width.
- Review: the adversarial gate scores the draft and returns a verdict, reasons, and fixes.
- Publish: only a PUBLISH verdict allows publish_draft to run, which shares the post to LinkedIn and records a real linkedin_urn.

7. Dashboard

All dashboard metrics are computed entirely from the product's own tables, never from LinkedIn engagement data, which the product does not have partner-level access to and does not pretend to have.

- Pillar coverage balance: distribution of ideas/drafts across the active pillars
- Substitutability trend: the substitutability score from review_results over time; lower is better
- Audience problem coverage: how much of the ICP's declared pains are addressed across pillars and ideas
- Stance advancement rate: published/reviewed drafts per stance over time

8. Security Model

- Row Level Security enforced on every table, either directly on user_id or traversed through the FK chain to brand_profiles.user_id
- publish_draft and the pillar/contrarian-gate triggers run as SECURITY DEFINER so the invariant holds even against direct API or SQL access
- The Groq API key lives only in Supabase secrets, scoped to the llm-gateway function
- .env is git-ignored from the first commit; no secrets are ever committed
- If LinkedIn OAuth is not configured in an environment, publishing fails with an explicit error; the product never simulates or fakes a successful publish

9. Explicitly Out of Scope

The following were deliberately cut and are not stubbed, placeholder-routed, or marked "coming soon" anywhere in the product: scheduling, carousels, image generation, a viral post library, engagement automation, auto-comments, auto-DMs, browser extensions, team seats, and client workspaces.

10. Design System

- Tone: restrained and editorial, a positioning tool, not a growth-hacking dashboard
- Palette: deep navy #0F1E2E, accent #1F6F8B, caution amber #A45C1E, pass green #2F6B4F, surface #F7F9FA, border #E4EAEE
- Typeface: Inter
- A rejected draft is shown in amber, never red; it is an editorial judgment, not a system error. Red is reserved for genuine errors.
- Explicit, purpose-built empty states for brand profile, pillars, ideas, and the dashboard, never a generic "no data" placeholder

11. Build Status

The build follows a locked six-phase plan, each phase ending at a demonstrable checkpoint before the next begins:

- Phase 1: schema, RLS, and auth (7 migrations applied)
- Phase 2: skeleton flow, routing, auth screens, and manual entry through the full profile > stance > pillar > idea > draft chain
- Phase 3: llm-gateway Edge Function deployed
- Phase 4: review gate wired to the adversarial system prompt, with publish_draft enforcing the verdict
- Phase 5: generation flows, including profile extraction from interview, pillar generation, and idea generation
- Phase 6: dashboard metrics and live LinkedIn publish/connect integration

All five Edge Functions (llm-gateway, review-draft, generate-content, publish-to-linkedin, linkedin-oauth-callback) are present and deployed, and all seven migrations are applied, indicating the full phase plan has been implemented.